

Bot Fraud Detection in Mobile Advertising

A modular implementation of the two detectors from the thesis *Bot Fraud Detection in Mobile Advertising*, built on the real **TalkingData AdTracking** dataset:

- **Algorithm 1 — Supervised:** feature-engineered **XGBoost** with class-imbalance handling, isotonic probability calibration, SHAP explanations, and an FPR-targeted operating threshold.
- **Algorithm 2 — Unsupervised:** a deep **autoencoder + Isolation Forest** that learns “normal” traffic, fuses reconstruction error with a latent-space anomaly score, and re-calibrates to track concept drift.

Both detectors share one 38-feature, 4-view representation (temporal, network/IP, device, conversion) and are compared on a baseline test set and across eight attack scenarios.

1. Repository layout

```
botfraud-detection/
├─ config.py                # all paths + hyperparameters (report Tables 3 & 4)
├─ requirements.txt
├─ data/                    # <-- put TalkingData train.csv here (you download
├─ src/
│   ├─ data_loader.py      # read real CSVs, chronological train/val/test spl
│   ├─ feature_engineering.py # the shared 38-feature, 4-view pipeline
│   ├─ algorithm1_xgboost.py # supervised detector
│   ├─ algorithm2_autoencoder.py # unsupervised detector (PyTorch AE + iForest)
│   ├─ scenarios.py        # 8 curated attack scenarios + training-set builde
│   ├─ evaluation.py       # metrics, ROC/PR/confusion plots, tables
│   └─ utils.py            # seeding, timing, normalisation, thresholding
├─ scripts/
│   ├─ run_all.py          # full pipeline end to end
│   ├─ run_algorithm1.py   # train + evaluate Algorithm 1 only
│   └─ run_algorithm2.py   # train + evaluate Algorithm 2 only
├─ tests/
│   └─ smoke_test.py       # generates a tiny schema-matching CSV; verifies t
├─ figures/                # generated plots
└─ results/                # generated metric tables (csv)
```

2. Setup

```
python -m venv .venv && source .venv/bin/activate      # optional
pip install -r requirements.txt
```

Tested with Python 3.12. PyTorch runs on CPU or GPU automatically; a GPU (e.g. your RTX-class card) speeds up Algorithm 2 but is not required.

3. Get the data

This project reads the **real** TalkingData corpus; it is not bundled (≈ 7 GB, ~ 184 M rows).

1. Download from Kaggle: *TalkingData AdTracking Fraud Detection Challenge*.

2. Place `train.csv` at: `botfraud-detection/data/train.csv`.

Raw schema: `ip, app, device, os, channel, click_time, attributed_time, is_attributed`.

By default the loader draws a reproducible ~ 5 M-row sample across the file (`config.DATA.sample_rows`) so it runs on a normal machine. Set it to `None` to use the whole corpus.

4. Run

```
python scripts/run_all.py           # data -> features -> both detectors -> evalua
python scripts/run_algorithm1.py    # supervised only
python scripts/run_algorithm2.py    # unsupervised only
```

Outputs: metric tables in `results/`, plots in `figures/` (`roc_baseline.png`, `pr_baseline.png`, `cm_algo1.png`, `cm_algo2.png`, `feat_importance.png`).

Verify the install without the download:

```
python tests/smoke_test.py
```

This writes a tiny CSV with the real TalkingData schema (and deliberately learnable signal) and runs the full pipeline. It is a correctness check only, not a results reproduction.

5. Design decisions you should be ready to defend

These are the methodological choices most likely to be challenged. Each is a deliberate, defensible decision — not an oversight.

5.1 Labelling & leakage (the most important point)

TalkingData ships only `is_attributed` (did a click convert to an install). The report's convention treats a **converting click as legitimate** and a **non-converting click as fraud**. That label is a *proxy*: "did not convert" is not the same as "is a bot," so it is an upper bound on true fraud, not ground truth. State this limitation explicitly.

Two consequences are handled in code:

1. **We do not train Algorithm 1 on the raw conversion flag.** Four features are derived from `attributed_time` (`has_install`, `time_to_install`, `tti_is_short`, `attribution_window_anomaly`). If the target were the raw conversion flag, `has_install` would be a *copy of the label* — textbook leakage that yields a meaningless AUC ≈ 1.0 . Instead, the supervised target is an **explicit, attribution-independent fraud label**: real background traffic labelled legitimate, plus injected attack patterns labelled fraud (the report's "attribution flag augmented by the curated scenario labels"). This keeps training and evaluation on the *same* notion of fraud and makes the post-attribution features safe to use. (If you ever switch the target back to the raw flag, set `EXCLUDE_POST_ATTRIBUTION=True` in `feature_engineering.py`.)
2. **All aggregate conversion-rate features are fit on the training split only** and mapped onto validation/test, so no future information leaks backwards (strict chronological discipline).

5.2 Synthetic scenarios

The eight attack scenarios are built by **synthetic injection** (volumetric bursts, device/IP concentration, short attribution windows, datacenter fan-out, a novel unseen-pattern variant). They measure behaviour against controlled, known patterns — they are **not confirmed real-world attack labels**. Treat the per-scenario numbers as a robustness probe, and be candid that the headline concept-drift comparison rests on a *simulated* novel variant; validating it on confirmed fraud is appropriate future work.

5.3 Concept drift is held out of supervised training

`concept_drift` is deliberately excluded from Algorithm 1's training mix (`scenarios.build_training_set(holdout=("concept_drift",))`), so it is genuinely novel at test time. This is what makes the supervised-vs-unsupervised comparison on unseen fraud

honest rather than circular.

5.4 Operating threshold

Thresholds target a fixed false-positive rate (`config.EVAL.target_fpr` , default 2%) rather than maximising F1, reflecting the deployment priority of not blocking genuine users. Read **ROC-AUC** as the threshold-free ranking quality and the precision/recall/F1 at the chosen operating point together.

5.5 The “datacenter/proxy” feature is a heuristic

TalkingData IPs are anonymised integers, so there is no real IP-reputation feed.

`datacenter_flag` is a **fan-out heuristic** (an IP spanning many device/OS combos). In production this would be replaced by an external reputation source; flag it as such.

6. Mapping to the report

Report element	Where it lives
38 features, 4 views (Table 2)	<code>feature_engineering.py</code>
XGBoost hyperparameters (Table 3)	<code>config.XGBConfig</code>
Autoencoder + iForest (Table 4)	<code>config.AEConfig</code> , <code>algorithm2_autoencoder.py</code>
Score fusion $s = \alpha \cdot \text{norm}(e) + (1-\alpha) \cdot \text{iForest}$	<code>AutoEncoderDetector._fuse</code>
Adaptive threshold / drift re-calibration	<code>AutoEncoderDetector.maybe_recalibrate</code>
8 scenarios (Table 1)	<code>scenarios.py</code>
Precision/Recall/F1/ROC-AUC/FPR/throughput	<code>evaluation.py</code>
Chronological split	<code>data_loader.py</code>

7. Honest limitations (good to pre-empt in the defense)

- Labels are a **conversion proxy**, not confirmed fraud (§5.1).

- Hard scenarios and the concept-drift finding rest on **synthetic injections** (§5.2).
- The only learned baseline is the other algorithm; adding a **published baseline** (e.g. a BotSpot-style graph model built from IP/device ↔ channel/app) would strengthen the comparison.
- `datacenter_flag` is a heuristic stand-in for IP reputation (§5.5).

The **hybrid cascade** (fast unsupervised screen → precise supervised confirmation) is the natural next step and the strongest candidate for a sharp, novel contribution; the two detectors here are the components it would combine.